

Desenvolvimento da Linguagem LIP Utilizando JFlex e Java CUP

1. Introdução

Este trabalho apresenta o desenvolvimento da linguagem **LIP** utilizando as ferramentas **JFlex** e **Java CUP** para a implementação de um analisador léxico e sintático.

O objetivo do projeto é demonstrar os conceitos fundamentais de construção de compiladores e interpretadores, incluindo análise léxica, análise sintática e execução de comandos definidos em uma linguagem própria.

A linguagem foi implementada em Java e possui suporte a variáveis, constantes, expressões aritméticas, estruturas condicionais, funções e comentários.

2. Ferramentas Utilizadas

Java

A linguagem Java foi utilizada para implementação do interpretador e integração entre os componentes gerados.

JFlex

O JFlex foi utilizado para a construção do analisador léxico (Scanner), responsável por transformar o código-fonte em uma sequência de tokens.

Java CUP

O Java CUP foi utilizado para implementação do analisador sintático (Parser), responsável por validar a estrutura do programa e executar as ações semânticas da linguagem.

3. Estrutura Geral da Linguagem

A linguagem LIP é composta por três etapas principais:

1. Leitura do código-fonte.
2. Análise léxica utilizando JFlex.
3. Análise sintática utilizando Java CUP.
4. Execução das ações associadas às produções da gramática.

O fluxo de execução pode ser representado da seguinte forma:

Código Fonte



Scanner (JFlex)



Tokens



Parser (Java CUP)



Execução

4. Análise Léxica

O analisador léxico identifica os elementos básicos da linguagem.

Palavras Reservadas

A linguagem reconhece as seguintes palavras reservadas:

if
else
while
return
const
var

Identificadores

Os identificadores são utilizados para nomear variáveis e funções.

Exemplos:

x
resultado
valor1
contador

Números

A linguagem suporta números inteiros.

Exemplos:

10
25
100

Operadores

Os seguintes operadores foram implementados:

Operador	Função
+	Soma
-	Subtração
*	Multiplicação
/	Divisão
=	Comparação e atribuição
,=	Diferente

Delimitadores

A linguagem utiliza:

(
)

para agrupamento de expressões e blocos.

Comentários

Comentários são delimitados utilizando:

<c>
conteúdo ignorado
<c>

Todo conteúdo entre os delimitadores é ignorado pelo analisador léxico.

5. Análise Sintática

A gramática da linguagem foi implementada utilizando Java CUP.

O símbolo inicial da gramática é:

programa

A produção principal é:

programa ::= lista_comandos

Cada programa é composto por uma lista de comandos.

lista_comandos ::= lista_comandos comando
 | comando

6. Variáveis

A declaração de variáveis é realizada utilizando a palavra reservada:

var

Exemplo:

var x

Após declarada, uma variável pode receber valores.

x = 10

Consultando o valor:

x

Saída:

Resultado da consulta: 10

7. Constantes

A linguagem suporta constantes utilizando a palavra reservada:

const

Exemplo:

const PI

Atribuição inicial:

PI = 3

Tentativa de alteração:

PI = 4

Saída:

Erro: constante não pode ser alterada: PI

8. Expressões Aritméticas

A linguagem suporta operações matemáticas básicas.

Soma

$10 + 2$

Resultado:

12

Subtração

$10 - 2$

Resultado:

8

Multiplicação

$10 * 2$

Resultado:

20

Divisão

$10 / 2$

Resultado:

5

Também são suportados parênteses para controle de precedência.

Exemplo:

$(10 + 2) * 2$

Resultado:

24

9. Estruturas Condicionais

A linguagem implementa a estrutura condicional:

```
if(condicao)
(
    comandos
)
else
(
    comandos
)
```

Exemplo:

```
if(2+2 = 4)
(
    1
)
else
(
    0
)
```

Saída:

1

Outro exemplo:

```
if(2-2 != 4)
(
    1
)
```

```
)  
else  
(  
  0  
)
```

Saída:

1

10. Funções

A linguagem suporta definição e execução de funções.

A declaração utiliza o caractere especial `$`.

Exemplo:

```
$funcSoma(a b)'a+b'
```

Nesta definição:

- `funcSoma` é o nome da função;
- `a` e `b` são os parâmetros;
- `a+b` é o corpo da função.

Chamada da função:

```
funcSoma(1 1)
```

Resultado:

2

Também é possível criar funções sem parâmetros.

Exemplo:

```
$funcTeste('$)'1+2+3'
```

Chamada:

```
funcTeste('$)
```

Resultado:

6

As funções são armazenadas internamente em uma tabela de funções e executadas por meio de reinterpretação do código armazenado.

11. Tratamento de Erros

A linguagem possui mecanismos básicos de tratamento de erros.

Variável não declarada

Exemplo:

```
y = 10
```

Saída:

Erro: identificador não declarado: y

Variável não inicializada

Exemplo:

```
var x
```

```
x
```

Saída:

Variável não inicializada ou expressão inválida.

Constante modificada

Exemplo:

```
const PI
```

```
PI = 3
```

```
PI = 4
```

Saída:

Erro: constante não pode ser alterada: PI

Comando inválido

Quando a entrada não corresponde a nenhuma produção válida da gramática, a linguagem informa:

Comando inválido.

12. Exemplo Completo

Exemplo de programa válido:

```
var x
```

```
var y
```

```
x = 10
```

```
y = 5
```

```
x + y
```

```
const PI
```

```
PI = 3
```

```
if(x)
```

```
(
```

```
    100
```

```
)
```

```
else
```

```
(
```

```
    0
```

```
)
```

```
$funcSoma(a b)'a+b'
```

```
funcSoma(1 1)
```

Saída aproximada:

Variável declarada: x

Variável declarada: y

Atribuição feita: x = 10

Atribuição feita: y = 5

Resultado da consulta: 15

Constante declarada: PI

Atribuição feita: PI = 3

Resultado da consulta: 100

Função criada: funcSoma

Resultado da consulta: 2

13. Conclusão

O projeto atingiu os objetivos propostos ao implementar uma linguagem interpretada utilizando JFlex e Java CUP.

Durante o desenvolvimento foram aplicados conceitos fundamentais de construção de compiladores, incluindo:

- análise léxica;
- análise sintática;
- tratamento de erros;
- tabelas de símbolos;
- execução de comandos;
- implementação de funções.

A linguagem resultante possui suporte a variáveis, constantes, expressões aritméticas, condicionais, comentários e funções, demonstrando a integração entre o analisador léxico gerado pelo JFlex e o analisador sintático gerado pelo Java CUP.

Link para o repositório: <https://github.com/luizing/lip---aspectos-teoriocos-da-computa-o>